

Independent A* Implementation Report

Overview

This notebook implements an independent A* path planner for multiple robots moving on a grid map. The planner computes one path per robot without considering the other robots during planning. That makes it a clean baseline for pathfinding performance, and it also makes conflict detection easy to analyze afterward.

The implementation is centered around the `IndependentAStarPlanner` class, which uses the `Node`, `GridEnvironment`, and `Robot` classes. The planner reads a text map, interprets walkable cells and obstacles, and then searches for the lowest-cost route from each robot's start position to its goal position.

Core Idea

A* combines:

- the exact cost from the start to the current state, called `g`
- a heuristic estimate from the current state to the goal, called `h`

The notebook uses Manhattan distance as the heuristic because movement is restricted to four directions: up, down, left, and right. For a grid with uniform movement cost, Manhattan distance is admissible and fits the environment naturally.

The total score for each node is:

$$f = g + h$$

The open set is stored as a min-heap, so the node with the smallest `f` value is expanded first.

Implementation Pieces

1. `Node`

The `Node` class stores the search state for A*:

- `position`: the current `(x, y)` location
- `parent`: the previous node in the path
- `g`: accumulated cost from the start
- `h`: heuristic cost to the goal
- `f`: total score used for prioritization

It also provides `__lt__`, `__eq__`, and `__hash__` so it can be used correctly in a priority queue and in sets or dictionaries. The `reconstruct_path()` method walks backward through the parent chain and returns the final route from start to goal.

2. `GridEnvironment`

This class loads the map from a text file and stores it as a 2D boolean grid:

- `.` becomes walkable space
- `T` becomes an obstacle

It exposes helper methods for:

- bounds checking
- walkability checks
- neighbor generation in the four cardinal directions
- Manhattan distance calculation

This keeps map logic separate from the search logic.

3. `Robot`

The `Robot` class represents one agent in the environment. It stores:

- robot id
- grid reference
- start position
- goal position
- current position
- planned path
- color for visualization

The planner uses the start and goal positions from each robot instance and returns a path keyed by robot id.

4. `IndependentAStarPlanner`

This is the main implementation of A*.

Initialization

The planner is created with a `GridEnvironment` instance. It also keeps a small cache so repeated start/goal queries can reuse previously computed paths.

```
run_independent_astar(robot, obstacles=None)
```

This method performs the actual search for one robot.

The algorithm flow is:

1. Read the robot's start and goal positions.
2. Reject invalid start or goal cells if either one is blocked.
3. Return immediately if start equals goal.
4. Initialize the open set with the start node.

5. Track `g_score` for best-known costs.
6. Repeatedly pop the node with the lowest `f` score.
7. Skip nodes already processed.
8. Stop when the goal is reached and reconstruct the path.
9. Otherwise expand neighbors from the grid.
10. Optionally allow waiting in place by appending the current position as a neighbor.
11. Update scores and push improved nodes back into the heap.
12. Return `None` if no route exists.

Important details:

- Movement cost is uniform and set to `1` per step.
- The search is limited to walkable grid cells.
- The `obstacles` parameter can block extra cells if needed, although the baseline independent planner usually runs without additional dynamic obstacles.

```
plan_all_robots(robots)
```

This method loops over all robots and runs A* separately for each one.

It also caches paths using a key built from:

- start position
- goal position
- obstacle configuration

That means identical queries can reuse stored results instead of recomputing them.

The important design choice is that robots are planned independently. The planner does not reserve space for other robots and does not perform coordination during search. That is why later validation and conflict detection are needed.

Conflict Detection Layer

The notebook includes a separate `ConflictDetector` class. It is not part of the A* search itself, but it is useful for evaluating the independent planner.

It checks for:

- vertex conflicts
- edge conflicts
- swap conflicts
- edge-swap conflicts
- following conflicts

This separation is intentional: A* finds shortest paths per robot, and the detector evaluates whether those paths collide when viewed together in time.

Validation Utilities

The notebook also includes helper functions to verify the output:

- `validate_paths()` checks that each path starts at the robot start, ends at the goal, stays inside the walkable map, and moves legally.
- `calculate_makespan()` returns the longest path length.
- `calculate_flowtime()` returns the sum of all path lengths.
- `count_conflicts()` uses `ConflictDetector` to count the total number of detected collisions.

These utilities are used to evaluate correctness and to compare runs.

Benchmark and Edge-Case Support

The notebook contains additional testing helpers for experiments:

- `run_benchmark()` runs the planner for different robot counts.
- `test_edge_cases()` tries scenarios such as shared starts, shared goals, dead ends, and blocked situations.

These are not required for the core A* algorithm, but they help show how the planner behaves under stress and special cases.

Swarm Demo and Visualization

The final notebook cell runs a swarm demo on a map file, currently using `../maps/large.txt`.

The demo does the following:

- loads the map
- finds walkable rows and columns dynamically
- picks 10 valid start positions and 10 valid goal positions
- assigns unique colors to the robots
- runs `plan_all_robots()`
- computes timing, makespan, flowtime, and conflict count
- renders both the grid and the resulting robot paths

The visualization is intentionally separate from the planner. That keeps the A* implementation clean and makes the notebook easier to reuse.

Why This Is an Independent A* Implementation

This notebook uses independent A* rather than cooperative planning. That means:

- each robot gets its own shortest path
- robots do not reserve time-space cells for each other

- planning is faster and simpler
- collisions may still happen in the combined solution

This makes the notebook well suited as a baseline implementation before introducing coordination strategies.

Practical Behavior

On the current large map, the swarm demo successfully plans 10 agents and reports:

- all paths found successfully
- zero detected conflicts in the current generated setup
- a visual plot of the maze and the computed routes

That shows the algorithm is working correctly on a large, obstacle-heavy grid and that the support code around it is stable.

Summary

The detailed implementation is built around a standard A* search with Manhattan heuristic, four-direction grid movement, and heap-based node expansion. The supporting classes isolate map handling and robot data, while conflict detection and validation are layered on top for analysis. The result is a clear baseline multi-robot pathfinding notebook with a reusable independent A* core.